MASTER IN AEROSPACE ENGINEERING

1MAE607 - Orbital Mechanics

Mission Analysis & Orbital Mechanics Hands-on Workshop (a.k.a. BE)

Auteurs :
Lapo Calosci

Encadrants :
Pr. Joan-Pau SANCHEZ CUARTIELLES

Version 0.1 du
March 18, 2025

Contents

Introduction **1**

1 Problem 1: Solving Kepler's Equation **3**

1.1 1a: Matlab function for Eccentric Anomaly using Newton-Raphson method 4

1.2 1b: Executing the Kepler's function 5

1.3 1c: MEO satellite orbit 6

1.4 1d: Analysis of eclipse times 8

2 Problem 2: Numerical integration of the Equations of Motions using `ode45` **13**

2.1 2a: 2-body equations in 3D 13

2.2 2b: The function *twobody* 14

2.3 2c: Function *twobody* and `ode45` 15

2.4 2d: Energies and Angular Momentum 18

2.5 2e: Polar orbit 20

3 Problem 3: Orbit Phasing and Rendezvous **25**

3.1 3a: Initial state vector of the ISS 26

3.2 3b: Interception after 12 orbits 28

3.3 3c: ΔV required to catch up 30

3.4 3d: De-orbiting the chaser 32

List of Figures

1.1 MEO satellite altitude as a function of time 8
1.2 Eclipses 9

2.1 Magnitude of position with respect to time 16
2.2 Magnitude of velocity with respect to time 17
2.3 3D orbit and Earth 17
2.4 Energies 18
2.5 Specific Angular Momentum 20
2.6 Magnitude of position in a day 21
2.7 Magnitude of velocity in a day 21
2.8 3D polar orbit and Earth 22

3.1 Initial situation for the chaser and the ISS 25
3.2 Initial vectors for the ISS 27
3.3 3d ISS 27
3.4 Distance between the chaser and the ISS 30
3.5 ΔV to catch up with the ISS in number of revolutions 32
3.6 ΔV required to reach a perigee 33

Introduction

The determination of a spacecraft's orbital parameters lies at the center of space missions and satellite operations, necessitating a thorough grasp of the dynamics governing celestial bodies. A fundamental problem in this domain is the Two-Body problem, which delves into the gravitational interaction between two elements, such as the Earth and a spacecraft. This report embarks on a practical implementation of algorithms to solve orbital mechanics questions.

We start our exploration by defining the essential equations governing the dynamics between a planet and a spacecraft. These equations are formulated under specific assumptions: firstly, we consider the spacecraft's mass as negligibly small compared to Earth. Secondly, an inertial coordinate system is adopted to facilitate our calculations. Thirdly, the Earth is approximated as a spherically symmetric body with uniform density, simplifying the calculations. Lastly, we assume that apart from the gravitational forces, no other external forces influence the system.

The fundamental equations used are:

$$\ddot{\mathbf{r}} + \frac{\mu_E}{r^3}\mathbf{r} = 0 \tag{1}$$

$$r = \frac{p}{1 + ecos(\theta)} \tag{2}$$

$$\frac{h^2}{\mu} = a(1 - e^2) = p \tag{3}$$

$$v = \sqrt{\mu(\frac{2}{r} - \frac{1}{a})} \tag{4}$$

$$r(\theta = 0) = r_p = \frac{p}{1 + e}; r(\theta = \pi) = \frac{p}{1 - e} \tag{5}$$

$$T = 2\pi\sqrt{\frac{a^3}{\mu}} \tag{6}$$

Where r and v are the radial distance and the velocity of the spacecraft, θ the true anomaly, p the semi latus rectum, a the semi-major axis, e the eccentricity, h the specific angular momentum, r_p and r_a the perigee and the apogee radii, and T the orbital period.

Chapter 1

Problem 1: Solving Kepler's Equation

In class, we have used the Kepler's Equation to couple the mean anomaly M, the Eccentric Anomaly E and the eccentricity e:

$$E - e\sin(E) = M = n(t - t_0) \tag{1.1}$$

$$n = \sqrt{\frac{\mu}{a^3}} = \frac{2\pi}{T} \tag{1.2}$$

where n is the mean motion, t the time, t_0 the time of periapsis passage and μ the gravitational parameter of the central body.

We also have a relation between the true anomaly and the eccentric anomaly:

$$\tan(\frac{E}{2}) = \sqrt{\frac{1-e}{1+e}}\tan(\frac{\theta}{2}) \tag{1.3}$$

Kepler's equation is transcendental and requires numerical methods for its solution. The Newton-Raphson is a root-finding algorithm that produces successively better approximations to the roots (or zeroes) of a real-valued function. In other words, it is an iterative method for solving equations of the form $f(x) = 0$ by successive approximations. By setting a chosen tolerance on the difference between the current solution and the next, it is possible to determine the convergence to a solution within the required accuracy.

More specifically, making an initial guess x_1 of the solution we want to find, we generate a sequence $x_1, x_2, x_3, x_4...$ using the formula:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)} \tag{1.4}$$

1.1 1a: Matlab function for Eccentric Anomaly using Newton-Raphson method

In our case, we will solve numerically equation (1.1) with the following step-by-step procedure:

1. pick a starting guess for E (in our case M);
2. calculate f(E) as: $f(E) = E - e\sin(E) + 1$;
3. calculate f'(E) as: $f'(E) = 1 - e\cos(E)$;
4. calculate the step increase: $\Delta E = -\frac{f(E)}{f'(E)}$
5. calculate the new E: $E_{k+1} = E_k + \Delta E$
6. repeat the previous steps until: $\mid \Delta E \mid < tol$

This procedure is implemented in the following function:

```
%% 1a
function [E,firstE,N_iter] = Kepler(M,e,tol)
    count = 0;
    Ek = M; % initial E is set as M
    while true
        count = count + 1;
        fk = Ek-e*sin(Ek)-M;
        dfk = 1 - e*cos(Ek);
        DE = - fk/dfk;
        Ek = Ek + DE; % increase the E
        if count==1 % save the first E
            firstE = Ek;
        end
        if abs(DE) < tol % check if DeltaE is less than tolerance
            E = Ek;
            N_iter = count; %save the number of iterations
            break;
        end
    end
end
```

1.2 1b: Executing the Kepler's function

In this part, I executed the Kepler function with M = 21° (0.3665 rad), $tol = 10^{-6}$ and $e = 0.25$, obtaining:

- After 1 iteration: $E = 0.48339$ rad
- Number of iterations to get under a tolerance of 10^{-6} is: 3
- After 3 iterations is: $E = 0.48252$ rad

By repeating the same calculations for $tol = 10^{-12}$, I obtain:

- After 1 iteration: $E = 0.48339$ rad
- Number of iterations to get under a tolerance of 10^{-12} is: 4
- After 4 iterations is: $E = 0.48252$ rad
- $\theta = 0.61515$ rad (calculated with equation (1.3))
- With $a = 24000$ km: $r = 18685.038$ km (calculated with equation (2))

By repeating the same calculations with M = 180° (π), the function finishes after the first iteration because $f(E_1) = \pi - e\sin(\pi) - \pi = 0$, that is obviously less than the tolerance. Thus with $tol = 10^{-6}$:

- After 1 iteration: $E = \pi$ rad
- Number of iterations to get under a tolerance of 10^{-6} is: 1

And the same for $tol = 10^{-12}$. True anomaly and radial distance are:

- $\theta = \pi$ rad
- With $a = 24000$ km, $r = 30000$ km

In this last case, as $E = M = \pi$, the radial distance is actually the apogee radius. Indeed, by using equation (5) we obtain the same result.

The code to obtain the previous results is:

```
%% 1b
clc;
M = 180;
M = M*pi/180;
e = 0.25;
tol = 1e-6;
[E,firstE,N_iter] = Kepler(M,e,tol);
disp(['Eccentric Anomaly after 1 iteration: ', num2str(firstE)
    , ' rad']);
disp(['Number of iterations to get under: ', num2str(tol), '
    is: ', num2str(N_iter)]);
disp(['Eccentric Anomaly after ',num2str(N_iter), ' iterations
    : ', num2str(E), ' rad']);

tol = 1e-12;
[E,firstE,N_iter] = Kepler(M,e,tol);
disp(['Eccentric Anomaly after 1 iteration: ', num2str(firstE)
    , ' rad']);
disp(['Number of iterations to get under: ', num2str(tol), '
    is: ', num2str(N_iter)]);
disp(['Eccentric Anomaly after ',num2str(N_iter), ' iterations
    : ', num2str(E), ' rad']);
true_anomaly = 2*atan(tan(E/2)*sqrt((1+e)/(1-e)));
disp(['True anomaly: ', num2str(true_anomaly), ' rad'])


a = 24000; %km
P = a*(1-e^2); %km
r = P/(1+e*cos(true_anomaly));
disp(['Radial distance with a=24000 km: ', num2str(r), 'km'])
    %km
```

1.3 1c: MEO satellite orbit

In this part, we were asked to compute every 15 seconds and plot the altitude of a MEO satellite by using the Kepler's function with the following information:

- t_0: spacecraft is at perigee

- Semi-major axis a = 24000 km
- Eccentricity e = 0.72

I implemented the following code:

```
a = 24000; % km
e = 0.72;
mu = 3.986*1e5;
n = sqrt(mu/((a)^3)); % mean motion
T = 2*pi*sqrt((a)^3/mu); %period
R = 6378; % km Earth radius
tol = 1e-6;

t_vector = []; %creating vectors
h_vector = [];
theta_vector = [];
E_vector = [];

for tk = 0:15:T
    Mk = n*tk;
    [E] = Kepler(Mk,e,tol);
    t_vector = [t_vector, tk]; %increasing the vectors
    h_vector = [h_vector, (a*(1-e*cos(E)) - R)];
    E_vector = [E_vector, E];
    theta_vector = [theta_vector, 2*atan(tan(E/2)*sqrt((1+e)
       /(1-e)))];
end

t_vector = t_vector*100/T;
plot(t_vector, h_vector, 'r')
grid on
title('Altitude in function of time')
xlabel('% of orbital period')
ylabel('Altitude in km')
```

where I have used, to calculate the radial distance, the equation:

$$r = a(1 - e\cos(E)) \tag{1.5}$$

that derives from equations (2), (3) and (1.3).

The plot that it generates is:

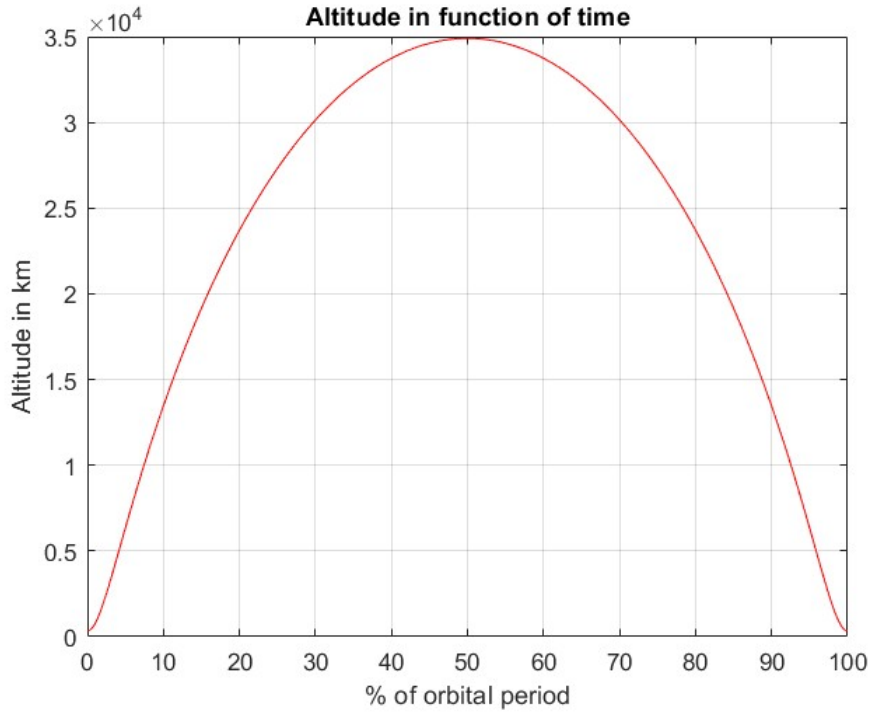

Figure 1.1: MEO satellite altitude as a function of time

1.4 1d: Analysis of eclipse times

In this exercise, we were asked to analyze two possible eclipses of the satellite along its MEO orbit due to Earth's shadow. Eclipes are fundamental to be known for the electric power organization of the satellite and thus, to know when it has to rely on batteries and how much energy we need to store before the eclipses.

In order to compute the points where the eclipses start and finish, we were provided with some formulae:

$$\alpha \cos^2 \theta + \beta \cos \theta + \gamma = 0 \tag{1.6}$$

where $\alpha = (R_E^2)(e^2) + p^2$, $\beta = 2R_E^2 e$ and $\gamma = R_E^2 - p^2$. The parameter p is calculated with equation (3). Solving equation (1.6), thanks to the `roots()` function, yields: $\theta_1 = 0.9166$ rad and $\theta_2 = 2.9813$ rad, which are two points of two different eclipses. By hypothesizing that the eclipses are symmetrical with respect to the line of apsides, we get another two true anomaly values: $(\theta_1)' = -0.9166$ rad and $(\theta_2)' = -2.9813$ rad. This completes the set of four angles needed to characterize the two eclipses.

The plot is shown in figure 1.2.

To get the plot, I implemented the following MATLAB code which makes use of what was calculated in section 1.3.

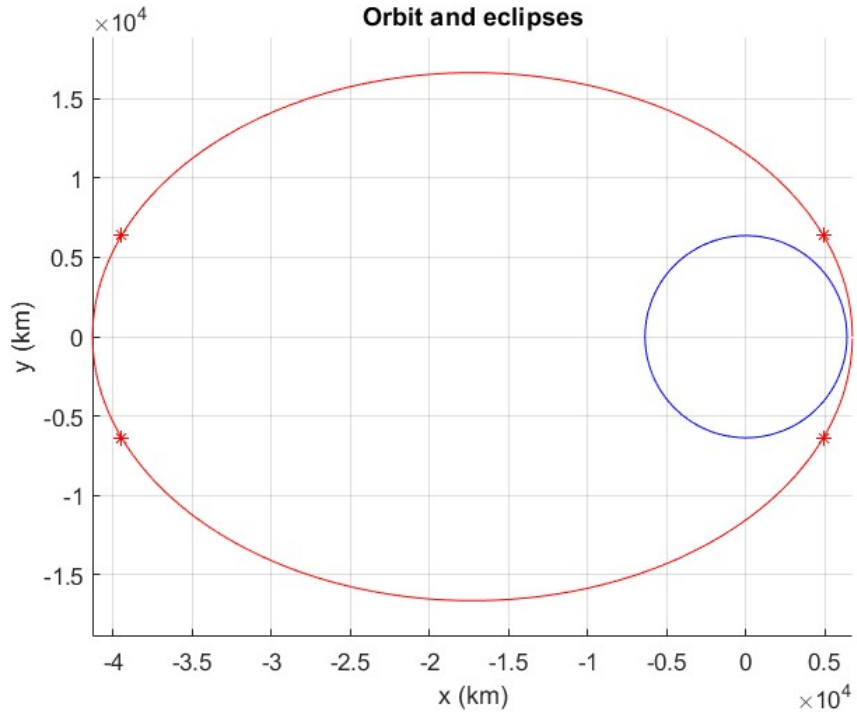

Figure 1.2: Eclipses

```
% 1d
r_vector = h_vector + R;

%calculating elliptic orbit points
x = r_vector.*cos(theta_vector);
y = r_vector.*sin(theta_vector);
%calculating "Earth plot"
th = 0:pi/50:2*pi;
x_circle = R * cos(th);
y_circle = R * sin(th);

figure
axis equal
grid on
hold on
plot(x,y,'r')
plot(x_circle, y_circle,'b')
title('Orbit and eclipses')
xlabel('x (km)')
```

```
ylabel('y (km)')

%calculating the formulas provided
p = a*(1-e^2);
alfa = (R^2)*(e^2)+p^2;
beta = 2*(R^2)*e;
gamma = R^2-p^2;
%I solve the trigonometric second order equation by using the roots
%function
v = [alfa beta gamma];
zeros = roots(v);
zeros = acos(zeros);

zeros = [zeros -zeros] %creating the simmetrical points with respect to the line of apsides
%plotting the points of start/finish of eclipses
for i= 1:4
    plot((p/(1+e*cos(zeros(i))))*cos(zeros(i)),(p/(1+e*cos(zeros(i))))*sin(zeros(i)),'r*')
end
hold off
```

To calculate the time spent in eclipse by the satellite, I used equation (1.1). In particular, since in such equation t_0 is the time of the perigee pass, I have written it in the following way:

$$\Delta t = t - t_0 = \frac{E - e \sin E}{n} \tag{1.7}$$

to calculate the time since perigee. For the first eclipse (the one happening "around" the perigee), I therefore simply injected the value of the E_1 (corresponding to θ_1) into the equation and I got half the time of the eclipse in seconds. For the second eclipse instead (the one starting at θ_2), I calculated the time to get from perigee to this angle and I subtracted this value from T/2. By doing so, I have:

- Time of the first eclipse: 23.0184 minutes, that corresponds to 3.7325% of the total period (616.7041 minutes);
- Time of the second eclipse: 131.2663 minutes, that corresponds to 21.2851% of the total period (616.7041 minutes)
- Total eclipse time: 154.2848 minutes

The time of the first eclipse is obviously less than the one of the second because the spacecraft is much faster around perigee. The code that calculates this is:

```
%calculating eclipse times
E1 = 2*atan((tan(zeros(2)/2)*sqrt((1-e)/(1+e)))) % zeros(2) corresponds to E = 0.9166 rad
Dt1 = (E1 - e*sin(E1))/n;
Dt1 = Dt1*2/60; %first eclipse time in minutes
disp(['Time of the first eclipse: ', num2str(Dt1),' minutes, that corresponds to ', num2str(Dt1/(T/60)*100),'% of the total period (', num2str((T/60)),' minutes) ']);

E2 = 2*atan((tan((zeros(1))/2)*sqrt((1-e)/(1+e)))); % zeros(1) corresponds to E = 2.9813 rad
Dt2 = T/2- (E2 - e*sin(E2))/n;
Dt2 = Dt2*2/60; %second eclipse time in minutes
disp(['Time of the second eclipse: ', num2str(Dt2),' minutes, that corresponds to ', num2str(Dt2/(T/60)*100),'% of the total period (', num2str((T/60)),' minutes) ']);
disp(['Total eclipse time: ', num2str(Dt1+Dt2), ' minutes'])
```

Chapter 2

Problem 2: Numerical integration of the Equations of Motions using `ode45`

The differential equations of the 2-body problem in 3 dimensions can be solved using a Matlab function called `ode45`. Given a time t and a state vector $\mathbf{X}$, this function returns the derivates of the state vector: $\dot{\mathbf{X}} = \mathbf{f}(\mathbf{t}, \mathbf{X})$.

2.1 2a: 2-body equations in 3D

In order to properly use `ode45` to solve the equations of motion in 3D, we have to write them in the form of:

$$\mathbf{X} = \begin{bmatrix} x \\ y \\ z \\ \dot{x} \\ \dot{y} \\ \dot{z} \end{bmatrix} = \begin{bmatrix} X_1 \\ X_2 \\ X_3 \\ X_4 \\ X_5 \\ X_5 \end{bmatrix} \qquad \dot{\mathbf{X}} = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{z} \\ \ddot{x} \\ \ddot{y} \\ \ddot{z} \end{bmatrix} = \begin{bmatrix} X_4 \\ X_5 \\ X_6 \\ f_1(X_1, X_2, X_3) \\ f_2(X_1, X_2, X_3) \\ f_3(X_1, X_2, X_3) \end{bmatrix} \tag{2.1}$$

We start from writing equation (1) ($\ddot{\mathbf{r}} + \frac{\mu_E}{r^3}\mathbf{r} = 0$) in its 3 components:

$$\begin{cases} \ddot{x} + \frac{\mu_E}{r^3}x = 0 \\ \ddot{y} + \frac{\mu_E}{r^3}y = 0 \\ \ddot{z} + \frac{\mu_E}{r^3}z = 0 \end{cases} \tag{2.2}$$

and from these three components, we derive the system of equations that can be used to

write on MATLAB the ode45 function:

$$\begin{cases} \dot{x} = v_x \\ \dot{y} = v_y \\ \dot{z} = v_z \\ \dot{v_x} = -\frac{\mu_E}{r^3} x \\ \dot{v_y} = -\frac{\mu_E}{r^3} y \\ \dot{v_z} = -\frac{\mu_E}{r^3} z \end{cases} \tag{2.3}$$

Thanks to this, we are able to write $\dot{\mathbf{X}}$ as:

$$\dot{\mathbf{X}} = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{z} \\ \ddot{x} \\ \ddot{y} \\ \ddot{z} \end{bmatrix} = \begin{bmatrix} X_4 \\ X_5 \\ X_6 \\ -\frac{\mu_E}{r^3} X_1 \\ -\frac{\mu_E}{r^3} X_2 \\ -\frac{\mu_E}{r^3} X_3 \end{bmatrix} \tag{2.4}$$

where $\mathbf{X}$ is the same as the one in equation (2.1) and $r = |\mathbf{r}| = \sqrt{X_1^2 + X_2^2 + X_3^2}$.

2.2 2b: The function *twobody*

From what has been written in the previous point, we have everything to create a function that describes the equations of motion:

```
%% 2a 2b
function dX = twobody(t,X)

    mu= 3.986*1e5; %km3/s2

    dX = [X(4);X(5);X(6); -mu/(sqrt(X(1)^2+X(2)^2+X(3)^2))^3*X
       (1); -mu/(sqrt(X(1)^2+X(2)^2+X(3)^2))^3*X(2); -mu/(sqrt
       (X(1)^2+X(2)^2+X(3)^2))^3*X(3)];

end
```

This function returns a column vector $\mathbf{f}$ corresponding to $\mathbf{f} = twobody(t, \mathbf{X})$.

2.3 2c: Function *twobody* and ode45

The ode45 has to be written in the form: [tout,yout]=ode45(@twobody, [t0:ts:tf], $\mathbf{X_o}$, options).
This code executes the integration of a system of differential equations found in *twobody* function from the initial time t_0 to the final time t_f, generating the solution at each time step t_s. It utilizes the initial state vector to specify the initial position and velocity. The initial state vector must be a column vector. In the options parameter, we can specify the error tolerances accepted for the numerical method.

As the function *twobody* contains a column vector with 6 equations, the function ode45 returns in yout a matrix where the columns correspond to the vector $\mathbf{X}$ and the rows to the times returned in tout.

In this specific part, we were asked to integrate the equations of motion of the 2-body problem for one day (84600 seconds) every 10 seconds, with a given initial vector $\mathbf{X_o}$ and a set of options, in order to calculate the magnitude of the position vector, of the velocity vector and the orbit in 3D. In this case, as the time steps are 8641, yout is a 8641x6 matrix.

To call the ode45 function I wrote:

```
r = [7115.804, 3391.696, 3492.221]; %km
v = [-3.762, 4.063, 4.184]; %km/s
options = odeset('RelTol',1e-12,'AbsTol', 1e-12);

[tout,yout]=ode45(@twobody, [0:10:86400], transpose([r,v]),
   options);
```

To plot the requested graphs, I wrote:

```
%plotting the position
figure(1)
abs_position = sqrt(yout(:,1).^2+yout(:,2).^2+yout(:,3).^2);
plot(tout/3600,abs_position,'r')
grid on
ylabel('Magnitude of position (km)')
xlabel('Time (h)')

%plotting the velocity
figure(2)
abs_velocity = sqrt(yout(:,4).^2+yout(:,5).^2+yout(:,6).^2);
plot(tout/3600,abs_velocity,'b')
grid on
ylabel('Magnitude of velocity (km/s)')
```

```
xlabel('Time (h)')

%plotting the Earth
R=6378; %km
figure(3)
hold on
plot3(yout(:,1),yout(:,2),yout(:,3))
title('3D orbit')
[X,Y,Z] = sphere;
s = surf(X*R,Y*R,Z*R);
set(s, 'FaceColor', [0 0 1])
axis equal
xlabel('x (km)')
ylabel('y (km)')
zlabel('z (km)')
hold off
```

and I obtained:

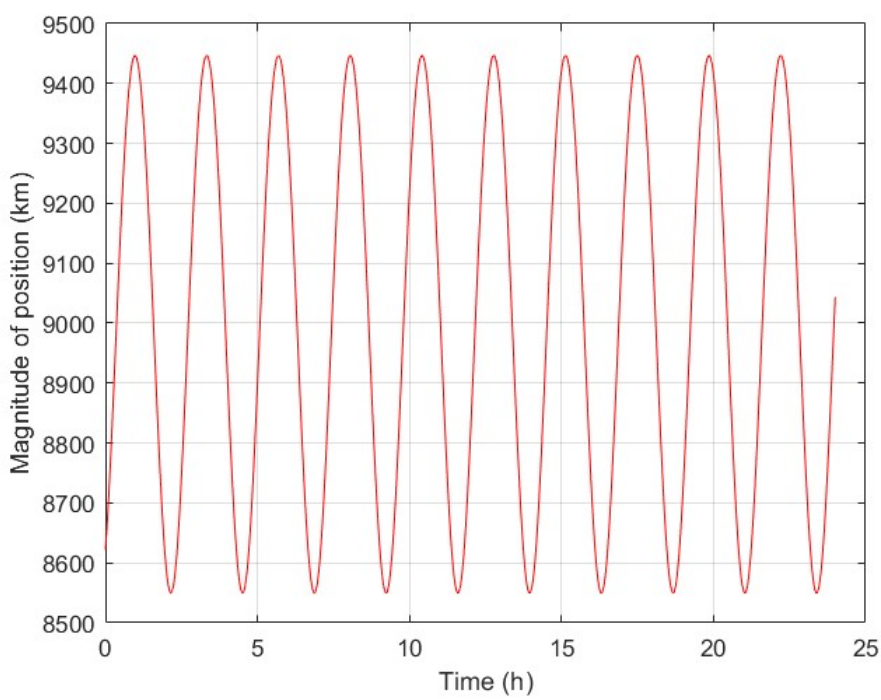

Figure 2.1: Magnitude of position with respect to time

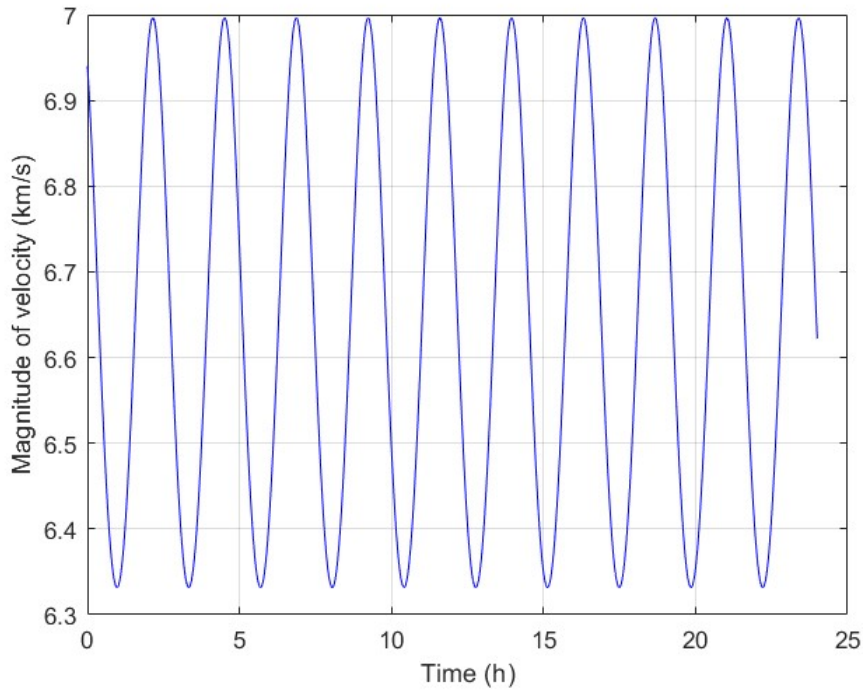

Figure 2.2: Magnitude of velocity with respect to time

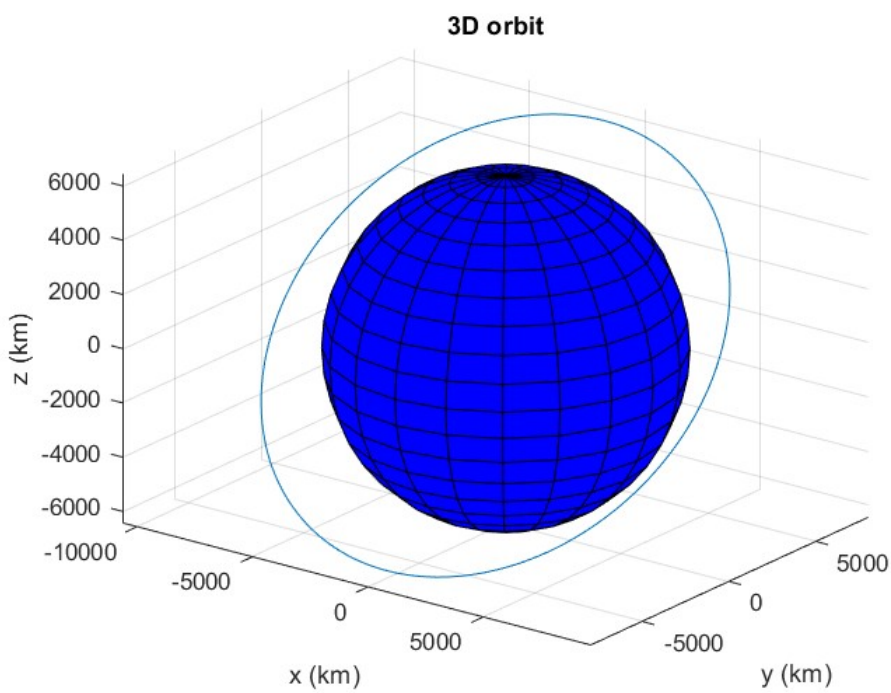

Figure 2.3: 3D orbit and Earth

2.4 2d: Energies and Angular Momentum

In this question, we were asked to calculate Specific Energies and Specific Angular Momentum. The equations to calculate, respectively, the specific kinetic and potential energies are the following:

$$U = -\frac{\mu_E}{r} \tag{2.5}$$

$$K = \frac{v^2}{2} \tag{2.6}$$

The total energy is:

$$E = U + K = -\frac{\mu_E}{r} + \frac{v^2}{2} \tag{2.7}$$

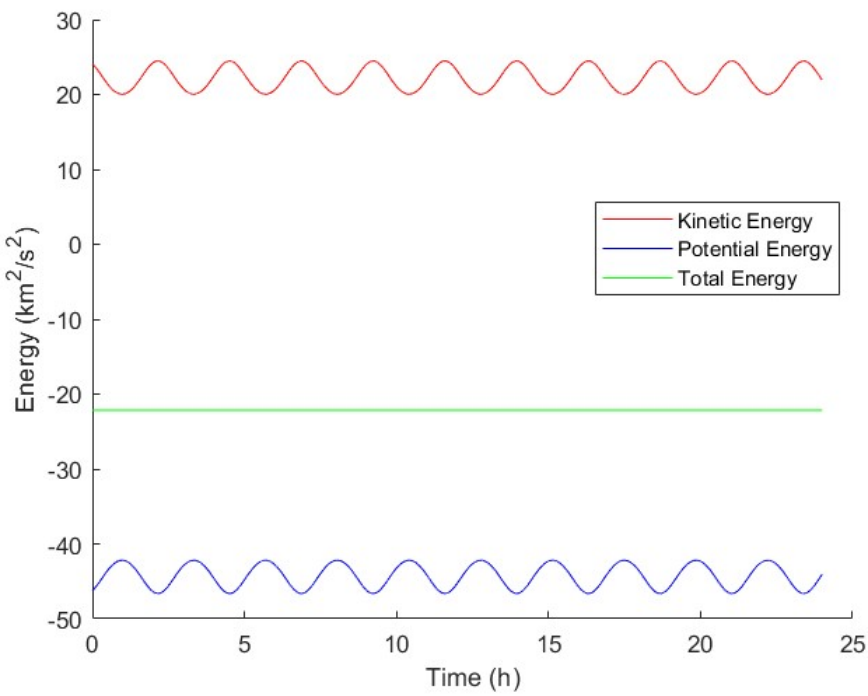

Figure 2.4: Energies

It is possible to observe in figure 2.4 that the kinetic and potential energies have an oscillatory behavior. Indeed, when the radius of the orbit diminishes, the velocity increases and thus the kinetic energy increases while the potential energy decreases (in the sense that it is more negative). Moreover, due to the fact that the gravitational field is a conservative one, the total energy is conserved and thus it remains constant along the orbit.

The code that I wrote to obtain such graphs is (it depends on the result of position and velocity obtained in section 2.3):

```
% plotting kinetic, potential and total energy
figure(4)
hold on
kinetic_E = (abs_velocity.^2)./2; %kinetic energy
plot(tout/3600, kinetic_E, 'r')

potential_E = -mu./abs_position; %potential energy
plot(tout/3600, potential_E, 'b')

total_E = kinetic_E + potential_E; %total energy
plot(tout/3600, total_E, 'g')
xlabel('Time (h)')
ylabel('Energy (km^2/s^2)')
legend('Kinetic Energy','Potential Energy', 'Total Energy')
hold off
```

The specific angular momentum is defined as the angular momentum of a body divided by its mass and can be written as:

$$\mathbf{h} = \mathbf{r} \times \dot{\mathbf{r}} \tag{2.8}$$

In the following MATLAB code, I calculated the components of the vectorial product and then I calculated the magnitude of **h**. By doing so I obtained the graph in figure 2.5. As it can be observed, the **h** is constant in time, and this is because its derivative is equal to zero, as it can be confirmed by the theory.

The code for specific angular momentum is:

```
%calculating specific angular momentum with vectorial product
h_x = ((yout(:,2).*yout(:,6))-(yout(:,3).*yout(:,5)));
h_y = ((yout(:,1).*yout(:,6))-(yout(:,4).*yout(:,3)));
h_z = ((yout(:,1).*yout(:,5))-(yout(:,4).*yout(:,2)));
h = sqrt((h_x).^2 + (h_y).^2 + (h_z).^2);

% plotting specific angular momentum
figure(5)
plot(tout/3600,h)
xlabel('Time (h)')
ylabel('Specific Angular Momentum (km/s^2)')
ylim([5.98*1e4 5.99*1e4]) %rescaling
```

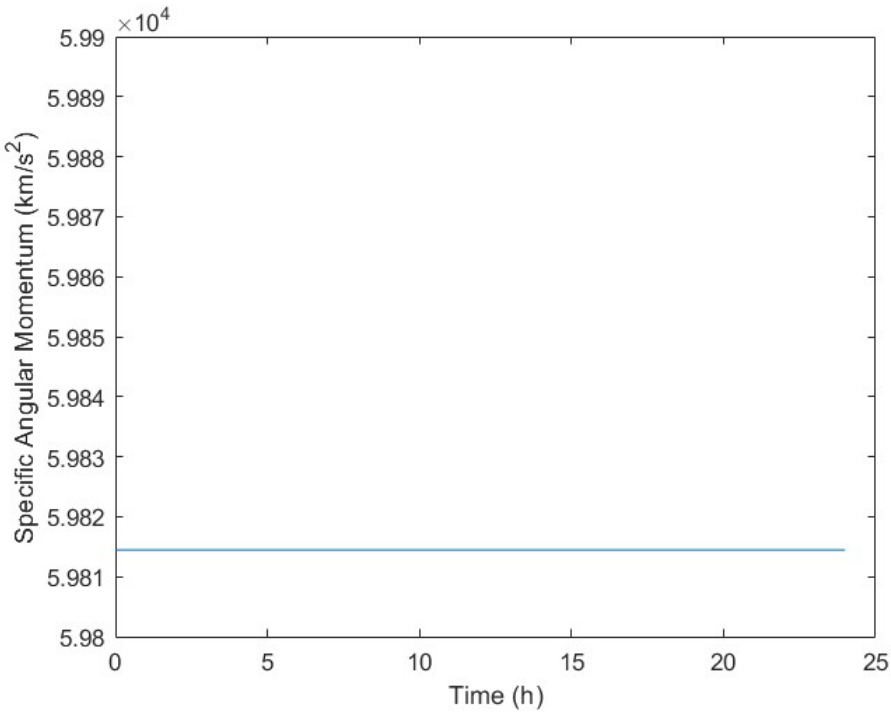

Figure 2.5: Specific Angular Momentum

2.5 2e: Polar orbit

In this point, we were asked to determine the type of orbit given a new initial state vector: $\mathbf{X_0} = (\mathbf{r}, \mathbf{v})^T$, with $\mathbf{r} = [0; 0; 8550]$ km, and $\mathbf{v} = [0; -7.0; 0]$ km/s.

For this orbit, I obtained the graphs in figures 2.6 and 2.7 for position and velocity.

It's thus clear that the orbit is elliptical as it shows changes in altitude and velocity with a periodic repetition. The semi-major axis of this new orbit is: 9009.9934 km and its period is: 141.8557 min (2 hours, 21 minutes and 51 seconds). Other hits and information are given by the following data:

- Energy of the new orbit: -22.1199 km^2/s^2
- Inclination of the new orbit is: 90°

The calculated orbital energy, as it is negative, is a clear indication of circular or elliptical orbit ($\epsilon = -\frac{\mu}{2a}$). If it was equal to zero, it would have been parabolic; and if it was more than zero, it would have been hyperbolic.
The inclination i is calculated with its definition: i is the angle between the $\mathbf{h}$ vector and the z axis, and it's therefore: $i = \arccos \frac{h_z}{h}$.

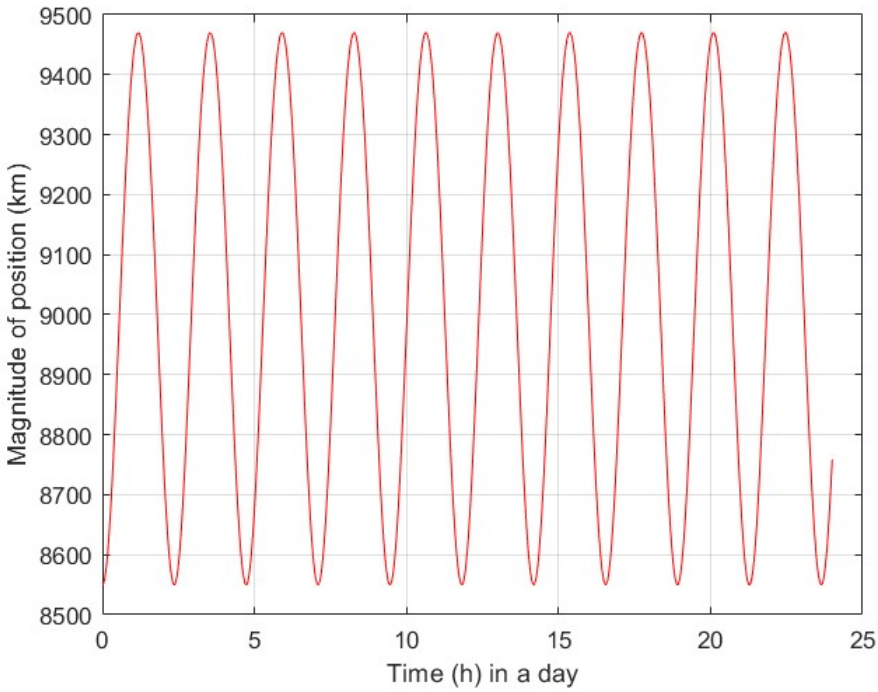

Figure 2.6: Magnitude of position in a day

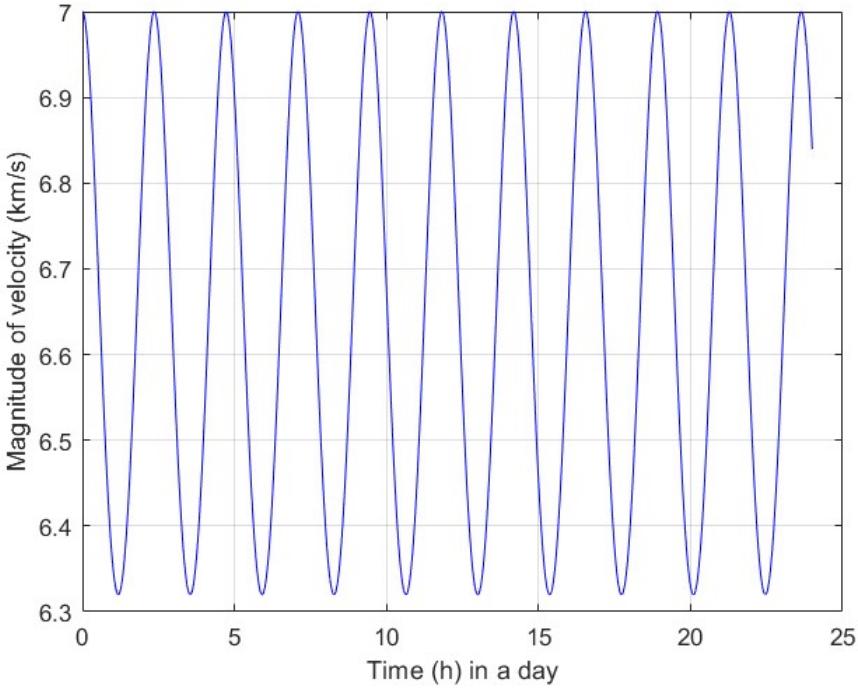

Figure 2.7: Magnitude of velocity in a day

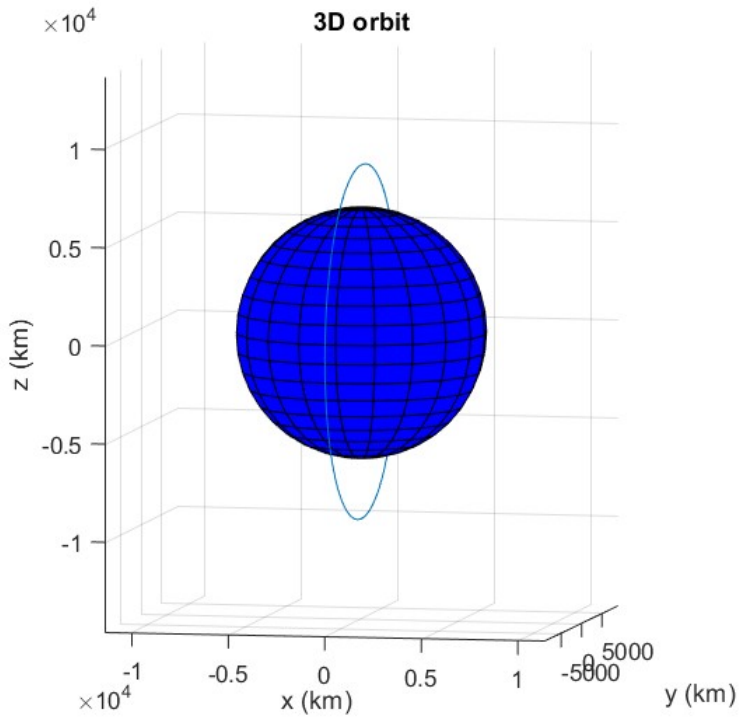

Figure 2.8: 3D polar orbit and Earth

The MATLAB code with which I obtained the previous results is:

```
r=[0, 0, 8550]; %km
v=[0, -7.0, 0]; %km/s
R=6378; %km
options = odeset('RelTol',1e-12,'AbsTol', 1e-12);
[tout,yout]=ode45(@twobody, [0:10:86400], transpose([r,v]),
   options);

figure(6)
abs_position = sqrt(yout(:,1).^2+yout(:,2).^2+yout(:,3).^2);
plot(tout/3600,abs_position,'r')
grid on
ylabel('Magnitude of position (km)')
xlabel('Time (h) in a day')

figure(7)
abs_velocity = sqrt(yout(:,4).^2+yout(:,5).^2+yout(:,6).^2);
plot(tout/3600,abs_velocity,'b')
grid on
```

```
ylabel('Magnitude of velocity (km/s)')
xlabel('Time (h) in a day')

%finding orbital parameters
r_perigee = min(abs_position);
r_apogee = max(abs_position);
%calculating semi-major axis
a = (r_apogee + r_perigee)/2;
disp(['Semi-major axis of the new orbit is: ', num2str(a),' km
    ']);
%calculating orbit period
T = 2*pi*sqrt((a^3)/mu);
disp(['Period of the new orbit is: ', num2str(T/60),' min']);
%calculating energy
energy = -mu/(2*a);
disp(['Energy of the new orbit: ', num2str(energy),' km^2/s^2'
    ]);
%calculating inclination from specific angular momentum
h_x = ((yout(:,2).*yout(:,6))-(yout(:,3).*yout(:,5)));
h_y = ((yout(:,1).*yout(:,6))-(yout(:,4).*yout(:,3)));
h_z = ((yout(:,1).*yout(:,5))-(yout(:,4).*yout(:,2)));
h = sqrt((h_x).^2 + (h_y).^2 + (h_z).^2);
inclination = acos((h_z(1))/h(1));
disp(['Inclination of the new orbit is: ', num2str(inclination
    *180/pi)]);

%plotting the 3d orbit and the Earth
figure(8)
hold on
plot3(yout(:,1),yout(:,2),yout(:,3))
title('3D orbit')
[X,Y,Z] = sphere;
s = surf(X*R,Y*R,Z*R);
set(s, 'FaceColor', [0 0 1])
xlabel('x (km)')
ylabel('y (km)')
zlabel('z (km)')
axis equal
grid on
hold off
```

Chapter 3

Problem 3: Orbit Phasing and Rendezvous

The third exercise concerns with determining the orbital parameters of a chaser cargo spacecraft (ex. Progess) for the ISS. In particular, we had to calculate the parameters needed to catch up and dock with the ISS, and to have the correct change in velocity to de-orbit the chaser and cause it to burn-up upon re-entry into the Earth's atmosphere.

The problem is simplified by putting the ISS on a circular equatorial orbit at an altitude of 404 km. The chaser is also placed in an equatorial orbit and it catches up the ISS during its own apogee: the altitude of the chaser at apogee is equal to the ISS altitude.

The initial situation is the one depicted in figure 3.1.

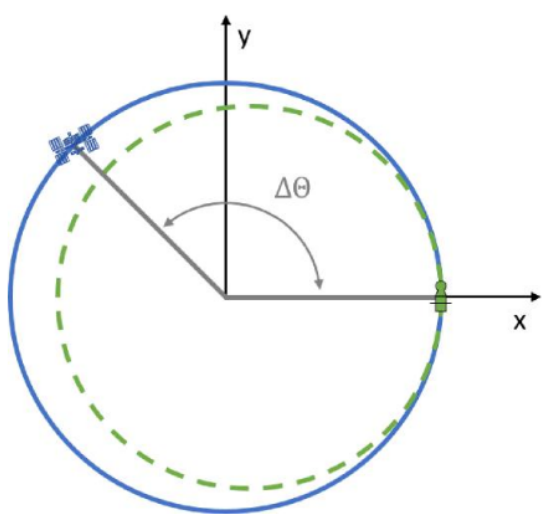

Figure 3.1: Initial situation for the chaser and the ISS

The angular separation is $\Delta\Theta$ and the ISS is ahead of such an angle from the chaser.

The chaser is put on a different orbit in order to have a shorter period than the ISS that will allow it to catch up with the ISS in N_{rev} orbits. This can be expressed with:

$$N_{rev}T_{chaser}(n_{chaser} - n_{ISS}) = \Delta\Theta \tag{3.1}$$

where T_{chaser} is the orbital period of the chaser and n_{chaser} and n_{ISS} are the mean motion of the chaser and the ISS respectively. Thanks to equation (3.1), we can calculate the relative angular velocity $n_{chaser} - n_{ISS}$ needed to make $\Delta\Theta$ going to zero in a $N_{rev}T_{chaser}$ interval of time. By using equation (6) and $n = \frac{2\pi}{T}$, it is possible to write:

$$a_{chaser} = (1 - \frac{\Delta\Theta}{2\pi N_{rev}})^{\frac{2}{3}} a_{ISS} \tag{3.2}$$

Finally, noting that the apogee of the chaser is equal to the orbital radius of the ISS, the value of the eccentricity of the chaser orbit can be calculated:

$$e_{chaser} = \frac{a_{ISS}}{a_{chaser}} - 1 \tag{3.3}$$

With these equations as instruments we can tackle the following points.

3.1 3a: Initial state vector of the ISS

In this point we were asked to calculate the initial state vector of the ISS using figure 3.1, and then plotting the orbit using `ode45` function.

In order to better understand how to calculate the initial state vector of the form $\mathbf{X_{0ISS}} = (\mathbf{r_{0iss}}, \mathbf{v_{0iss}})^T$ (necessary to use the `ode45` function), let's have a look at figure 3.2.
From such image, we understand that the components of the radial distance and of the velocity vectors can be calculated as follows:

$$\mathbf{r_{0iss}} = [r\cos(\Delta\Theta), r\sin(\Delta\Theta), 0] = [-1177.7, 6679, 0]km \tag{3.4}$$

$$\mathbf{v_{0iss}} = [-v\cos(\Delta\Theta - \pi/2), -v\sin(\Delta\Theta - \pi/2), 0] = [-7.5499, -1.3313, 0]km/s \tag{3.5}$$

Where $r = 404km + R_E$ and $v = \sqrt{(\frac{\mu_E}{r})}$ from the Vis-Viva equation (4).
By using equation (6), we find the period of the ISS equal to: 92.6396 minutes or 1 hours 32 minutes and 38 seconds.

The 3d orbit is in figure 3.3.

The code to calculate all these results follows.

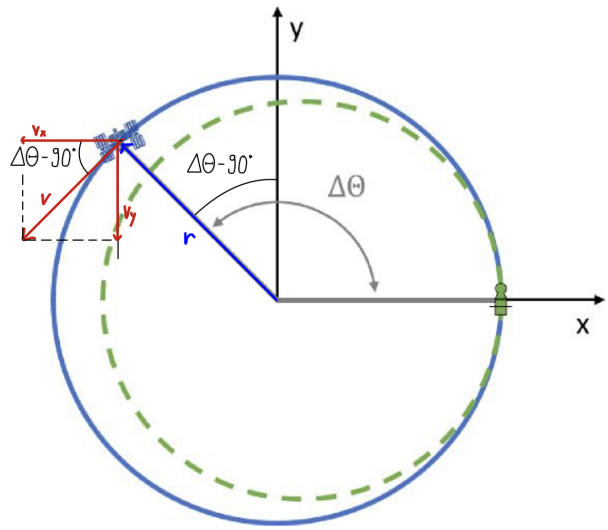

Figure 3.2: Initial vectors for the ISS

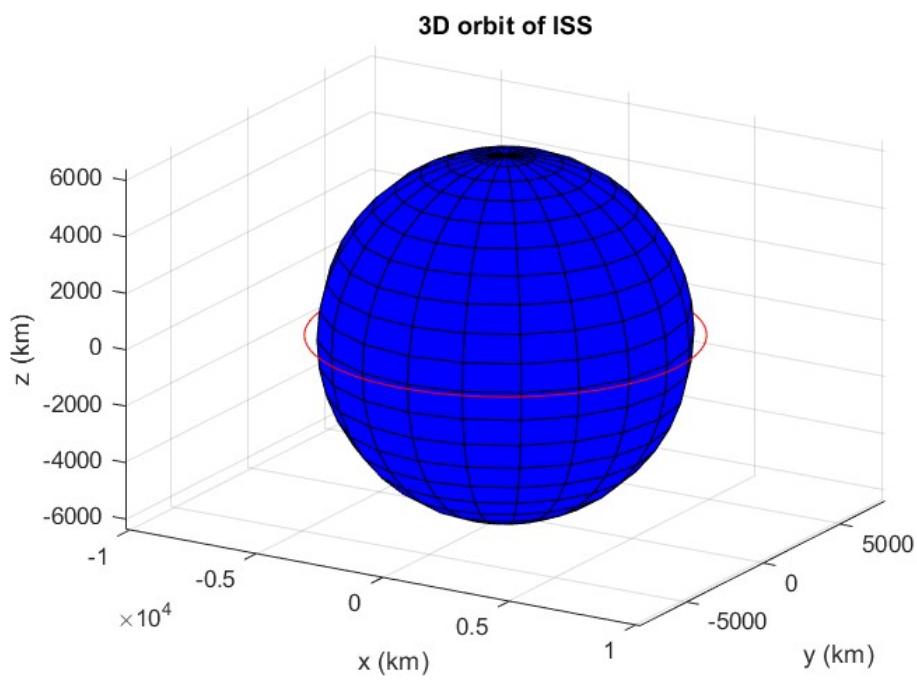

Figure 3.3: 3d ISS

```
h_iss = 404; %km
R = 6378; %km
mu= 3.986*1e5; %km3/s2
```

```
D_Theta = 100; % degrees

magnitude_r = h_iss+R;
r0_iss = [(magnitude_r)*cos(D_Theta*pi/180), (magnitude_r)*sin
   (D_Theta*pi/180), 0] %km

magnitude_v = sqrt(mu/magnitude_r); % circular orbit
v0_iss = [- magnitude_v*cos((D_Theta-90)*pi/180), -magnitude_v
   *sin((D_Theta-90)*pi/180), 0] %km/s

%calculating the orbit in a day
options = odeset('RelTol',1e-12,'AbsTol', 1e-12);
[tout,yout] = ode45(@twobody, [0:10:86400], transpose([r0_iss,
   v0_iss]), options);
%plotting the 3d orbit of the Iss and the Earth
figure(1)
hold on
plot3(yout(:,1),yout(:,2),yout(:,3))
title('3D orbit of ISS')
[X,Y,Z] = sphere;
s = surf(X*R,Y*R,Z*R);
set(s, 'FaceColor', [0 0 1])
xlabel('x (km)')
ylabel('y (km)')
zlabel('z (km)')
axis equal
grid on
hold off

%calculating orbital period
T_iss = 2*pi*sqrt(magnitude_r^3/mu);
disp(['Iss orbit period in minutes is: ', num2str(T_iss/60), '
    minutes'])
```

3.2 3b: Interception after 12 orbits

With the use of equations (3.2) and (3.3), we can calculate the eccentricity and the semi-major axis of the elliptical orbit of the chaser to intercept the ISS after 12 orbits (N_{rev}), and from these derive its initial state vector $\mathbf{X_{0chaser}} = (\mathbf{r_{0chaser}}, \mathbf{v_{0chaser}})^T$ and its orbital

period. The values obtained are: $a_{chaser} = 6676.9315$ km and $e_{chaser} = 0.015736$.

Then, by noting that the initial position of the chaser is at the apogee (the ISS altitude), I easily derived the $\mathbf{r_{0chaser}} = [r_{iss}, 0, 0] = [6782km, 0, 0]$.

To get the velocity vector we need to use the Vis-viva equation (4) and inject in it the just found value of the semi-major axis of the chaser. By doing so, I derived the vector: $\mathbf{v_{0chaser}} = [0, 7.6058, 0]$ km/s.
The following code calculates these results:

```
N_rev = 12; %number of revolution to catch up with ISS

%semi-major axis and eccentricity
a_iss = h_iss+R;
a_chaser = (1-(D_Theta*pi/180)/(2*pi*N_rev))^(2/3)*a_iss;
e_chaser = a_iss/ a_chaser -1;
disp(['Semi-major axis of the chaser: ', num2str(a_chaser), '
   km'])
disp(['Eccentricity chaser: ', num2str(e_chaser)])

%chaser initial vector
r_apogee_chaser= a_iss;
r0_chaser = [r_apogee_chaser,0,0];
v0_chaser = [0, sqrt(mu*(2/r_apogee_chaser - 1/a_chaser)), 0];
    %velocity on elliptical orbit
```

Another request was to plot the distance from the chaser to the ISS as the former catches up. I calculated this distance by calculating the orbit points with `ode45` for both the spacecrafts and then I obtained the distance with:

$$distance = \sqrt{(x_{iss} - x_{chaser})^2 + (y_{iss} - y_{chaser})^2 + (z_{iss} - z_{chaser})^2} \tag{3.6}$$

The following MATLAB code allows to get figure 3.4.

```
%orbits and distance
T_chaser = 2*pi*sqrt(a_chaser^3/mu);
[tout_chaser,yout_chaser] = ode45(@twobody, [0:5:(T_chaser*
   N_rev)], transpose([r0_chaser,v0_chaser]), options);
[tout,yout] = ode45(@twobody, [0:5:(T_chaser*N_rev)],
   transpose([r0_iss,v0_iss]), options);

```

```
distance = sqrt((yout(:,1)-yout_chaser(:,1)).^2+(yout(:,2)-
   yout_chaser(:,2)).^2+(yout(:,3)-yout_chaser(:,3)).^2);

%plotting the distance at each revolution
figure(2)
hold on
plot(tout_chaser/T_chaser, distance, 'r')
title('Distance at each orbit')
ylabel('Distance in km')
xlabel('Number of revolutions')
grid on
hold off
```

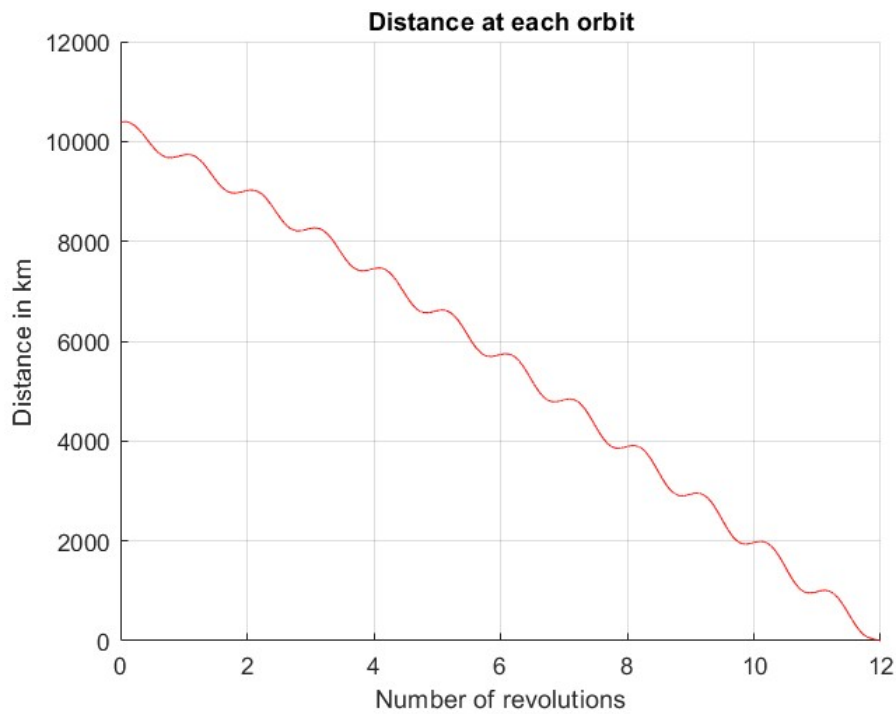

Figure 3.4: Distance between the chaser and the ISS

3.3 3c: ΔV required to catch up

The chaser will also need a ΔV to match its apogee speed to the ISS's one before it can dock. Indeed, it can be demonstrated that at the apogee of the elliptical orbit of the chaser $v_{ISS} = \sqrt{(\frac{\mu}{r_{apogee}})} > v_{chaser} = \sqrt{\mu(\frac{2}{r_{apogee}} - \frac{1}{a})}$, and thus the chaser requires to perform a

ΔV to circularize its orbit.
Thus, by using these formulas we have:

$$\Delta V = v_{ISS} - v_{chaser} = \sqrt{(\frac{\mu}{r_{apogee}})} - \sqrt{\mu(\frac{2}{r_{apogee}} - \frac{1}{a})} \tag{3.7}$$

This exercise required calculating the ΔV to match the apogee of the chaser to the speed of the ISS before it docks for each N_{rev} in the interval of N_{rev}=[2:30]. For each value of N_{rev}, the semi-major axis, the eccentricity and the period of the chaser need to be recalculated with the procedure described in section 3.2. This new data is used in the equation (3.7) to calculate the ΔV at each iteration.

By implementing the following code:

```
N_rev = [2:30]; %number of revolution to catch up with ISS
D_Theta = 100; % degrees
h_iss = 404; %km
R = 6378; %km
mu= 3.986*1e5; %km3/s2

a_iss = h_iss+R;
magnitude_v_iss = sqrt(mu/a_iss); % circular orbit
r_apogee_chaser = a_iss;

for i =1:29
    a_chaser = (1-(D_Theta*pi/180)/(2*pi*N_rev(i)))^(2/3)*
       a_iss;
    e_chaser = a_iss/ a_chaser -1;

    %orbits and distance
    T_chaser = 2*pi*sqrt(a_chaser^3/mu);

    v_chaser_apogee(i) = sqrt(mu*(2/r_apogee_chaser - 1/
       a_chaser));
    D_V(i) = magnitude_v_iss - v_chaser_apogee(i);
end

plot(N_rev, D_V)
grid on
title('\DeltaV to catch up the ISS in number of revolutions ')
xlabel('Number of revolutions')
ylabel('\DeltaV (km/s)')
```

I obtained the following graph:

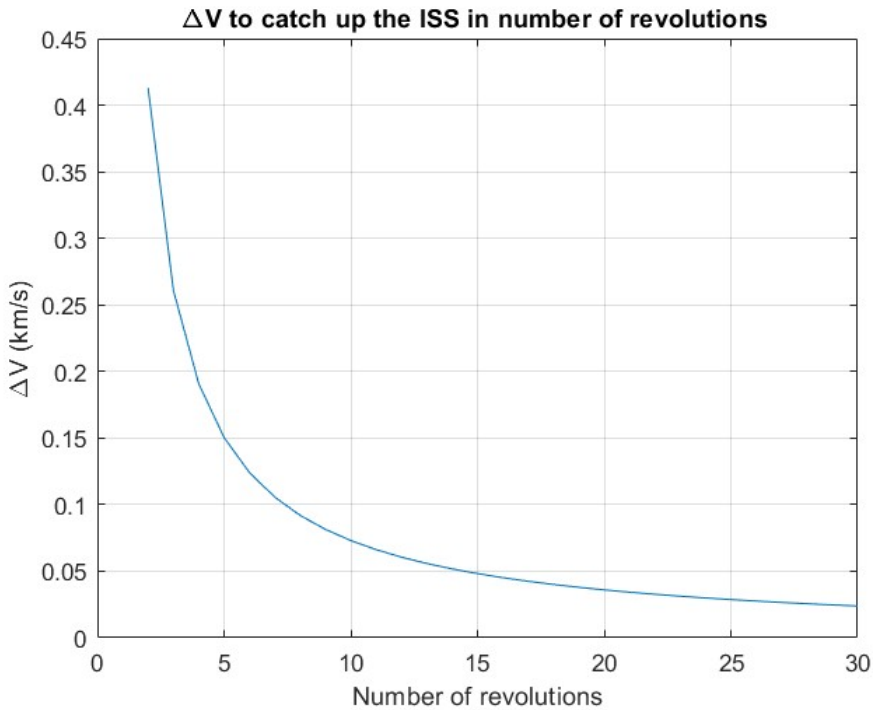

Figure 3.5: ΔV to catch up with the ISS in number of revolutions

As it can be observed in figure 3.5, the higher (lower) is the number of revolutions to catch up, the lower (higher) is the ΔV required. How can this be explained? The explanation is the following: the less the number of orbits for the chaser to catch up is, the faster it has to cover its orbit to reach the ISS on time and to make $\Delta\Theta$ going to zero. An orbit covered in less time ("faster orbit") is generally an orbit with a lower semi-major axis (as stated by the third Kepler's law (6)). An elliptical orbit with a lower semi-major axis is an orbit that at the apogee has a lower velocity. Indeed, the term $\frac{1}{a}$ in the Vis-Viva equation gets higher as a gets lower. By having a smaller apogee velocity (and by the fact that the ISS velocity is always unchanged), the chaser has to generate more ΔV to circularize its orbit once it has reached the ISS. This therefore explains why we need more ΔV to circularize the orbit when the number of revolutions is lower.

3.4 3d: De-orbiting the chaser

In the last point of this report, it was asked to calculate and plot the ΔV required to lower the perigee of the Progress capsule to the range of values 60 km to 210 km while keeping the apogee unchanged. De-orbiting a cargo spacecraft is a critical point of a re-supply mission

to the ISS, in that the point of re-entry has to be calculated precisely to not impair people or things on the ground.

The higher the perigee of the re-entry orbit, the more unpredictable the point of re-entry is due to the unpredictability of the behavior, in the atmospheric burn-up process over a long period of time, of an object not made to survive re-entry. This uncertainty derives from not having a precise enough model of the behavior of the atmosphere, which implies a significant inaccuracy in the s/c position over a long period of time (indeed the higher the perigee, the longer the time to re-enter is). Therefore, the perigee altitude has to be thoughtfully decided.

The graph that I obtained is in figure 3.6.

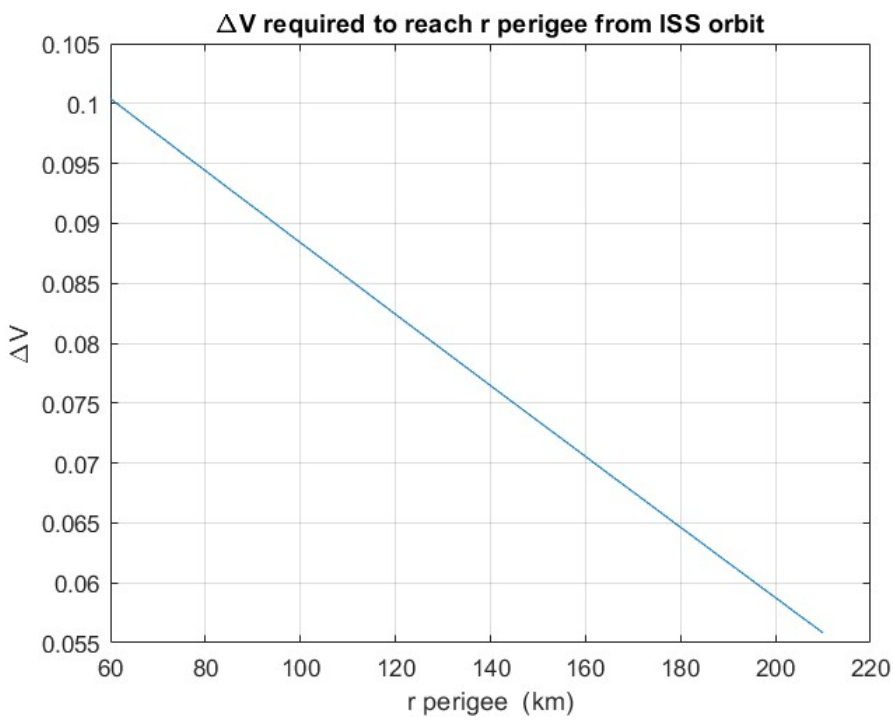

Figure 3.6: ΔV required to reach a perigee

This result was obtained with the following MATLAB code:

```
h_iss = 404; %km
R = 6378; %km
mu= 3.986*1e5; %km3/s2

a_iss = h_iss + R;
magnitude_v_iss = sqrt(mu/a_iss); % circular orbit
r_apogee_chaser = a_iss;
```

```

h_perigee =[60:0.5:210]; %km

for i = 1:301
    a_new(i) = (r_apogee_chaser+h_perigee(i)+R)/2;
    D_V_reenter(i) = abs(-magnitude_v_iss+sqrt(mu*(2/
       r_apogee_chaser - 1/a_new(i))));
end

plot(r_perigee, D_V_reenter)
grid on
xlabel('r perigee  (km)')
ylabel('\DeltaV')
title('\DeltaV required to reach r perigee from ISS orbit')
```

In this code, I simply created an interval of $r_{perigee}$ and I calculate the new semi-major axis by using: $a = \frac{r_{apogee}+r_{perigee}}{2}$. This new a is used to calculate the velocity of apogee for the chaser in its re-entry elliptical orbit. By looking at figure 3.6, it can be understood that the lower the altitude we want the chaser to reach, the more the required ΔV is. Indeed, to get to a lower altitude the cargo spacecraft has to decelerate more, and this can be again understood from the Vis-Viva equation. It tells us that, fixed the apogee altitude, the elliptical orbit with the lowest velocity at the apogee is the one with the lowest semi-major axis (the one with the lower altitude reached at perigee).

It is important not to think that the graph in figure 3.6 has a linear form. Indeed, this is only a visual bias due to the fact that the nonlinearity of the trend (given by the presence of the squared root in the Vis-Viva equation) is perceptible only on a larger span of the x value. In other words, the nonlinearity is "slow" and locally the graph resembles a straight line.

An important observation has to be made about the sign of the ΔV. In theory, the calculations of the ΔV give us a negative value, because it is $\Delta V = v_{chaser} - v_{ISS} = \sqrt{\mu(\frac{2}{r_{apogee}} - \frac{1}{a})} - \sqrt{(\frac{\mu}{r_{apogee}})}$ (that is basically the calculus used for Homann Transfer Orbit when we have to reach a smaller orbit than the initial one). However, the negative sign of ΔV would not make sense if it was used in the rocket equation: $\Delta V = v_e \ln \frac{m_0}{m_f}$, because it would imply that the initial mass is lower than the final mass: the spacecraft would gain mass when burning fuel. Therefore, the only concrete meaning of a negative ΔV is that it refers to a retrograde burn, i.e. a burn made in the opposite direction of the orbital velocity vector.